COMPUTER ENGINEERING

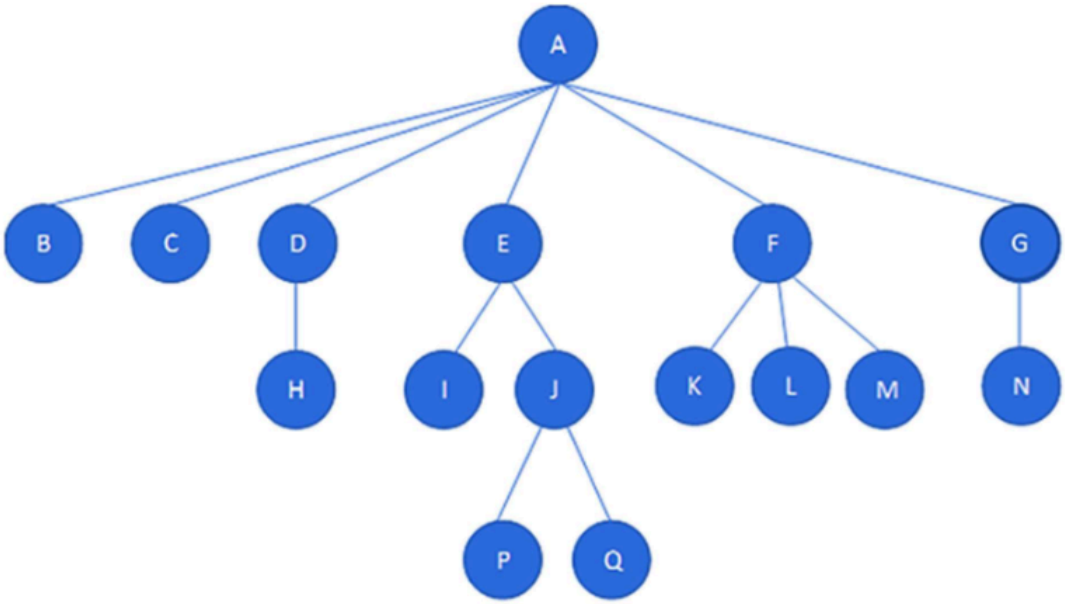
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	09/24/2026	**DATE SUBMITTED:**	09/24/2026
NAME:	Baltazar, Kenji	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork No. 9.1 **Implementing Trees**

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.



Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. Indicate your code screenshot and comments in table 9-1

CODE *(Tree.h)*

```cpp
#ifndef TREE_H
#define TREE_H

template<typename T>
class Tree{
private:
    T key;
    Tree<T>* firstChild;
    Tree<T>* nextSibling;

public:
    Tree(T value);
    void insertChild(T value);

    Tree<T>* getFirstChild();
    Tree<T>* getNextSibling();
    T getRootVal();
};

template<typename T>
Tree<T>::Tree(T value){
    key = value;
    firstChild = nullptr;
    nextSibling = nullptr;
}

template<typename T>
void Tree<T>::insertChild(T value){
    Tree<T>* newNode = new Tree<T>(value);
```

```
31      if(firstChild == nullptr){
32          firstChild = newNode;
33      }else{
34          Tree<T>* current = firstChild;
35          while(current->nextSibling != nullptr){
36              current = current->nextSibling;
37          }
38          current->nextSibling = newNode;
39      }
40  }
41
42  template<typename T>
43  Tree<T>* Tree<T>::getFirstChild(){
44      return firstChild;
45  }
46
47  template<typename T>
48  Tree<T>* Tree<T>::getNextSibling(){
49      return nextSibling;
50  }
51
52  template<typename T>
53  T Tree<T>::getRootVal(){
54      return key;
55  }
56
57  #endif
```

CODE ***(procedure.cpp)***

```cpp
#include <iostream>
#include "tree.h"

int main(){
    Tree<char>* root = new Tree<char>('A');

    //under A
    root->insertChild('B');
    root->insertChild('C');
    root->insertChild('D');
    root->insertChild('E');
    root->insertChild('F');
    root->insertChild('G');

    //children D
    Tree<char>* D = root->getFirstChild()->getNextSibling()->getNextSibling();

    D->insertChild('H');

    //children E
    Tree<char>* E = D->getNextSibling();

    E->insertChild('I');
    E->insertChild('J');

```

```
26      //children J
27      Tree<char>* J = E->getFirstChild()->getNextSibling();
28
29      J->insertChild('P');
30      J->insertChild('Q');
31
32      //children F
33      Tree<char>* F = E->getNextSibling();
34
35      F->insertChild('K');
36      F->insertChild('L');
37      F->insertChild('M');
38
39      //children G
40      Tree<char>* G = F->getNextSibling();
41      G->insertChild('N');
42
43      return 0;
44  }
```

OBSERVATION

Using the binaryTree.h header file, the code was transformed to be more suited for general trees where the connections under the root is not only limited to left and right children but can have multiple siblings.
So, the tree was created by first making a root A, then adding all its children from B to G. Then after all these nodes were created, the same goes to the childrens of B to G, we just connect them to each nodes of one another to form a tree.

Task 2: Complete the following table:

NODE	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1

F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Pre-Order A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-Order B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A
In-Order B, A, C, H, D, I, E, P, J, Q, K, F, L, M, N, G

CODE *(Tree.h)*

```
21      void preOrder();
22      void postOrder();
23      void inOrder();
```

```cpp
template<typename T>
void Tree<T>::preOrder(){
    std::cout << key << " ";
    Tree<T>* current = firstChild;
    while(current != nullptr){
        current->preOrder();
        current = current->nextSibling;
    }
}
```

```cpp
template<typename T>
void Tree<T>::inOrder(){
    if(firstChild != nullptr){
        firstChild->inOrder();
    }
    std::cout << key << " ";
    Tree<T>* current = nullptr;
    if(firstChild != nullptr){
        current = firstChild->nextSibling;
    }
    while(current != nullptr){
        current->inOrder();
        current = current->nextSibling;
    }
}
```

```cpp
template<typename T>
void Tree<T>::postOrder(){
    Tree<T>* current = firstChild;
    while(current != nullptr){
        current->postOrder();
        current = current->nextSibling;
    }

    std::cout << key << " ";
}

```

CODE ***(procedure.cpp)***

```cpp
    std::cout << "Pre-order: ";
    root->preOrder();

    std::cout << "\nPost-order: ";
    root->postOrder();

    std::cout << "\nIn-order: ";
    root->inOrder();
```

OUTPUT

```
Pre-order: A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order: B A C H D I E P J Q K F L M N G
```

OBSERVATION

Using the same file, I added 3 more functions in the header file for pre,post,inOrder. This function mainly uses the concept of recursion meaning that the functions call itself. For the pre Order, the root will be displayed first then it will go down to its children, though there is no left and right distinction as it is not binary. For in Order, the first child is outputted which is B then followed by the root, then the remaining ones. For the post Order, all the children will be outputted first before the root.

CODE *(Tree.h)*

```
101  template<typename T>
102  bool findData(Tree<T>* root, int choice, T value){
103      if(root == nullptr)
104          return false;
105
106      if(choice == 1){
107          if(root->getRootVal() == value)
108              return true;
109
110          Tree<T>* current = root->getFirstChild();
111
112          while(current != nullptr){
113              if(findData(current, choice, value))
114                  return true;
115
116              current = current->getNextSibling();
117          }
118      }else if(choice == 2){
119          Tree<T>* current = root->getFirstChild();
120
121          while(current != nullptr){
122              if(findData(current, choice, value))
123                  return true;
124
125              current = current->getNextSibling();
126          }
127
128          if(root->getRootVal() == value)
129              return true;
130      }else if(choice == 3){
```

```
131         Tree<T>* current = root->getFirstChild();
132 
133         if(current != nullptr){
134             if(findData(current, choice, value))
135                 return true;
136 
137             if(root->getRootVal() == value)
138                 return true;
139 
140             current = current->getNextSibling();
141 
142             while(current != nullptr){
143                 if(findData(current, choice, value))
144                     return true;
145 
146                 current = current->getNextSibling();
147             }
148         }else if(root->getRootVal() == value){
149             return true;
150         }
151     }
152     return false;
153 }
```

CODE ***(procedure.cpp)***

```
52     if(findData(root, 1, 'K'))
53         std::cout << "K was found!" << std::endl;
54 
55     if(findData(root, 2, 'E'))
56         std::cout << "E was found!" << std::endl;
57 
58     if(findData(root, 3, 'N'))
59         std::cout << "N was found!" << std::endl;
```

OUTPUT

```
E was found!
N was found!
```

OBSERVATION

This find data function uses the traversal choice I made as a condition. These choices determine what kind of order traversal will be used, then after choosing, the loop will then check the values in the node whilst traversing. Then once the data has been found then it will be outputted.

CODE *(procedure.cpp)*

```
39      //children G
40      Tree<char>* G = F->getNextSibling();
41      G->insertChild('N');
42      G->insertChild('O');
```

```
62      if(findData(root, 3, 'O'))
63          std::cout << "O was found!" << std::endl;
```

OUTPUT

```
O was found!
```

OBSERVATION

This is for adding a new leaf node O, which is created it as children of G. Then we just point it from G and output it using the findData.

2. ANSWER TO SUPPLEMENTARY ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

CODE

```cpp
#include <iostream>
#include "BST.h"

int main(){
    BinarySearchTree<int> tree(2);

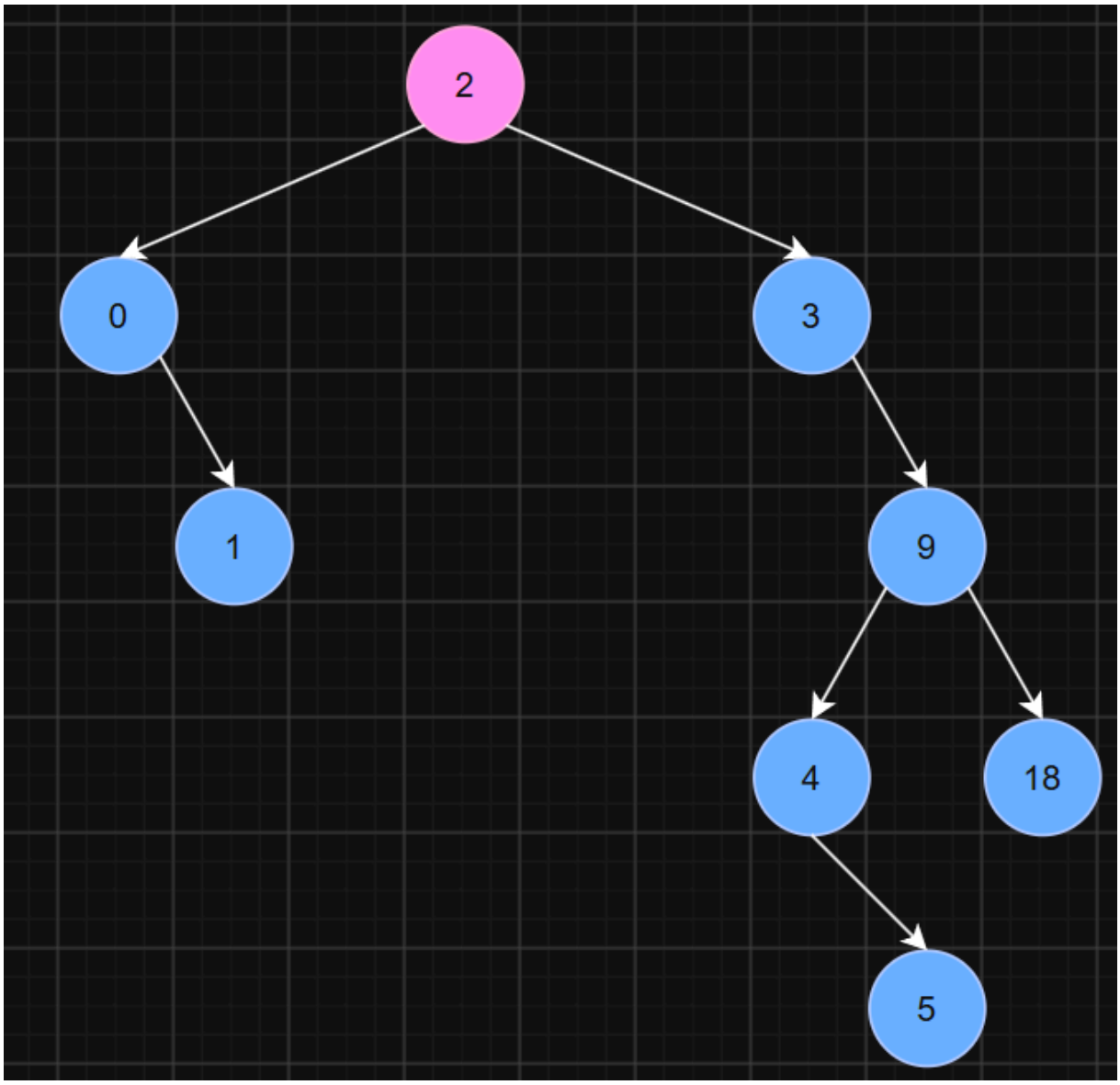
    std::cout<<"Pre Order: ";
    tree.insert(2);
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    tree.preOrder();
    std::cout<<"\nIn Order: ";
    tree.inOrder();
    std::cout<<"\nPost Order: ";
    tree.postOrder();

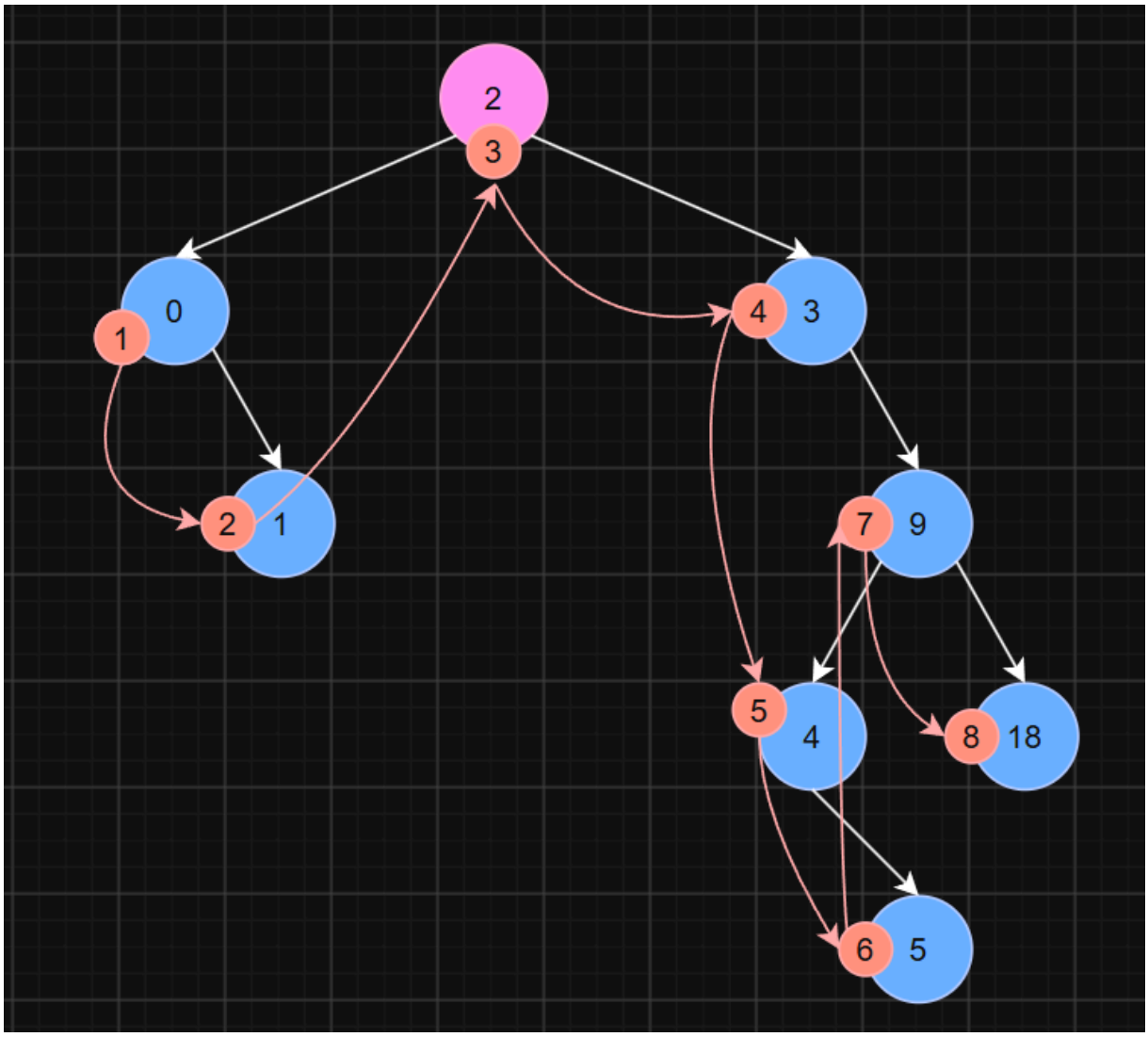
    return 0;
}
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

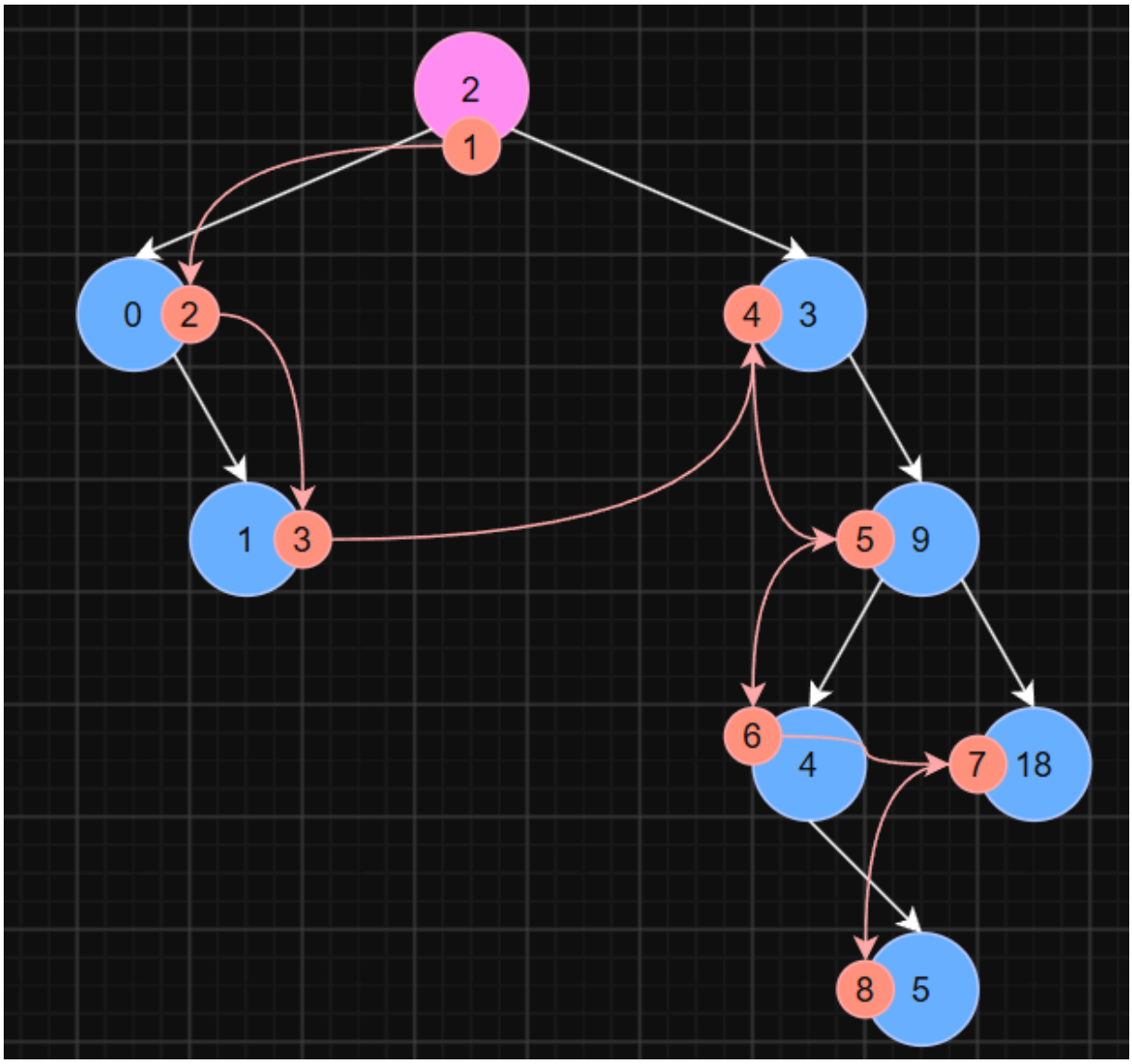
TREE DIAGRAM



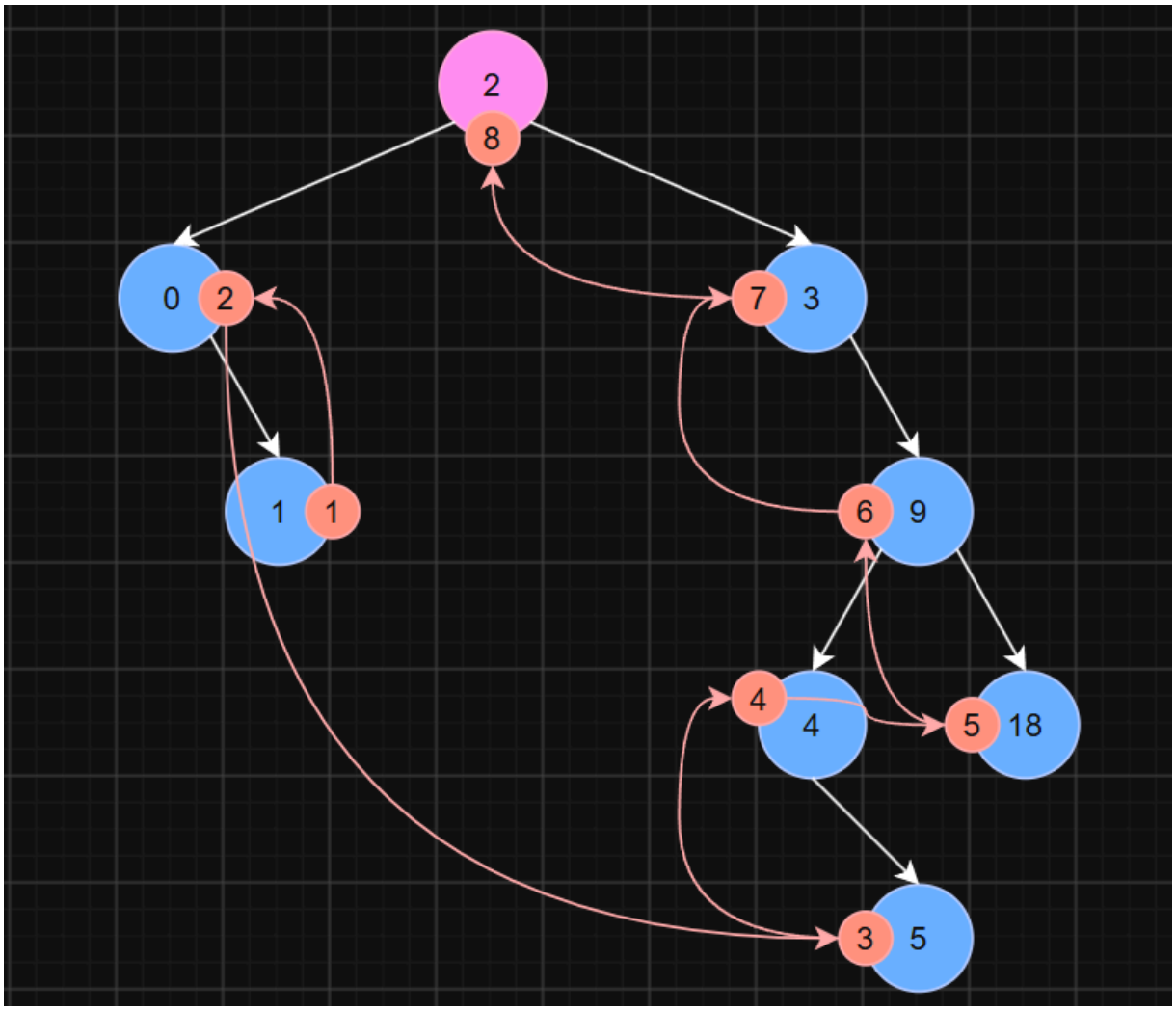
IN ORDER



PRE ORDER



POST ORDER



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

FUNCTION

```
template <typename T>
void BinarySearchTree<T>::inOrder(){
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<<key<< " ";
    if(right != nullptr){
        right->inOrder();
    }
}
```

OUTPUT

```
In Order: 0 1 2 3 4 5 9 18
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

FUNCTION

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<<key<< " ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

OUTPUT

```
Pre Order: 2 0 1 3 9 4 5 18
```

OBSERVATION

Nope, the output is not different from what I had predicted in the diagram, as the order starts from the root, going to the left hand child going down to check left which is null and outputs the right. It then goes to the right hand side and does the same thing. ROOT -> LEFT -> RIGHT.

Post-order Traversal

FUNCTION

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<<key<< " ";
}
```

OUTPUT

```
Post Order: 1 0 5 4 18 9 3 2
```

OBSERVATION:

Nope, the output is not different what my prediction, the Order starts at the very left bottom of the left hand child of the root which is Null, then it outputs the right side and goes up to output the left hand side. It then goes to the right hand child and goes to the most left under it and the similar process goes on until it reaches the root. LEFT->RIGHT->ROOT

3. CONCLUSION

In this activity, I learned a lot about trees and traversing. I first learned about trees where it has many types but the one first is the general tree which can have more than 2 children of its own, then binary tree which can only have up to a maximum of 2 children. I also learned about height and depth/level; height is from leaf to root, counting from 0. depth/level starts from the root to the leaf, counting from 0 as well. I also learned a lot about the traversing methods used, which are Pre-order, Post-order, In-order. For pre-order, it follows the ROOT->LEFT->RIGHT principle where it first prints the first most number then goes down left and then right. For Post-order it follows the LEFT->RIGHT->ROOT principle where it first starts at the very left leaf then traverses its way up until the level 1 part of the tree and goes to the right hand side most bottom left leaf. Then for the In order, it just follows a triangular

pattern and follows the LEFT->ROOT->RIGHT principle where the main idea is that it can produce or output a segment with a distinguishable in order pattern. Overall, I learned a lot in this hands on and I can say I am quite well versed in terms of binary tree but I am still lacking in the general tree aspect regarding the coding. The concepts as well as the analysis of where goes to where is also ingrained deeply which I am proud of.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used only in procedure to help me transform the binaryTree.h header file suited for a general tree. But it is not used in observational analysis, supplementary, and the conclusion are all my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.